

ELECTRONICS GUIDE | جزوهٔ آموزشی مهندسی الکترونیک

استفاده از رابط SDIO
در حالت Slave برای سری CH32
با نمونهٔ عملی CH32V407

از مرور ویژگی‌های کنترل‌کننده (Controller) رابط SDIO و دیاگرام اتصالات سخت‌افزاری میزبان (Host) و دستگاه پیرو (Slave)، تا روش پیکربندی رجیسترها، سازوکار پاسخ‌دهی به فرمان‌ها و روند کامل ارسال و دریافت داده با DMA

در این جزوه، ترجمهٔ فارسی سند رسمی AN21000 (نسخهٔ ۱٫۰، منتشرهٔ شرکت WCH) را می‌خوانید: رابط SDIO میکروکنترلر به‌عنوان دستگاه پیرو (Slave) پیکربندی می‌شود تا به فرمان‌های یک SDIO Host (میزبان خارجی) پاسخ دهد و داده جابه‌جا کند. اتصال دو CH32V407 در نقش میزبان و دستگاه پیرو، جدول نگاشت پایه‌ها، نکات اتصال سخت‌افزاری، تنظیم ساعت و پهنای گذرگاه، پاسخ خودکار R1، پیکربندی DMA و روند کامل ارسال و دریافت، همگی با فهرست‌های واقعی کد مرور می‌شوند.

میکروکنترلر CH32V407 · رابط SDIO در حالت Slave · CMD و پاسخ R1 · انتقال داده با DMA
پهنای گذرگاه ۱، ۴ و ۸ بیتی · پیکربندی رجیسترها · نمونهٔ میزبان–دستگاه پیرو · واژه‌نامهٔ فنی

ترجمهٔ فارسی AN21000 V1.0 · نسخهٔ ۱٫۰ — آبان ۱۴۰۵

فهرست مطالب — برای پرش به بخش موردنظر، روی ردیف آن کلیک کنید؛ شمارهٔ صفحه‌ها بر اساس صفحات فیزیکی PDF است.

فهرست مطالب

۱ توضیحات و دامنهٔ کاربرد ۳

۲ بخش ۱ | مرور رابط SDIO در سری CH32 ..... ۳

1.1 مرور کلی ۳

۳ بخش ۲ | نمونهٔ طراحی کامل حالت SDIO Slave ۴

2.1 اتصال سخت‌افزاری ۴

2.1.1 اتصال پایه‌ها ۵

2.2 پیکربندی رجیسترها ۵

2.2.1 پیکربندی میزبان ۵

2.2.2 پیکربندی دستگاه پیرو ۸

۴ بخش ۳ | آزمون و راستی‌آزمایی ۱۰

۵ تاریخچهٔ نسخه‌ها و بیانیه ۱۱

۶ واژگان کلیدی (واژه‌نامهٔ فنی) ۱۱

فهرست شکل‌ها، جدول‌ها و کدها

شکل ۱-۱ دیاگرام ارتباط میزبان و دستگاه پیرو SDIO ۴

جدول ۲-۱ تخصیص پایه‌های سیگنال ۵

کد ۱ مقداردهی اولیهٔ پایه‌ها، ساعت و کنترل‌کنندهٔ SDIO در میزبان ۶

کد ۲ ارسال فرمان 0x05 و انتظار پاسخ در میزبان ۷

کد ۳ پیکربندی DMA و تابع ارسال داده در میزبان ۷

کد ۴ مقداردهی اولیهٔ دستگاه پیرو و فعال‌سازی حالت SLV ۸

کد ۵ پیکربندی پاسخ R1 و انتظار فرمان در دستگاه پیرو ۹

کد ۶ پیکربندی DMA و تابع دریافت داده در دستگاه پیرو ۹

چکیده —

این یادداشت کاربردی (Application Note) نحوه استفاده از رابط SDIO در تراشه‌های سری CH32 را توضیح می‌دهد؛ به‌گونه‌ای که این رابط در حالت Slave (دستگاه پیرو) پیکربندی شود تا بتواند به فرمان‌های ارسال‌شده از سوی یک SDIO Host (میزبان خارجی) پاسخ دهد و عملیات انتقال داده را انجام دهد.

در این مقاله، تراشه CH32V407 به‌عنوان نمونه مورد بررسی قرار گرفته است. ابتدا ویژگی‌ها و قابلیت‌های کنترل‌کننده (Controller) رابط SDIO معرفی می‌شوند. سپس یک نمونه طراحی کامل از حالت SDIO Slave ارائه می‌شود که شامل موارد زیر است: دیاگرام اتصالات سخت‌افزاری (Hardware Connection Diagram)؛ روش پیکربندی رجیسترها (Register Configuration)؛ سازوکار پاسخ‌دهی به فرمان‌ها (Command Response Mechanism) و روند ارسال و دریافت داده (Data Transmission and Reception).

مقدمه

توضیحات و دامنهٔ کاربرد

این یادداشت کاربردی (Application Note) نحوه استفاده از رابط SDIO در تراشه‌های سری CH32 را توضیح می‌دهد؛ به‌گونه‌ای که این رابط در حالت Slave (دستگاه پیرو) پیکربندی شود تا بتواند به فرمان‌های ارسال‌شده از سوی یک Host (میزبان خارجی) پاسخ دهد و عملیات انتقال داده را انجام دهد.

در این مقاله، تراشه CH32V407 به‌عنوان نمونه مورد بررسی قرار گرفته است. ابتدا ویژگی‌ها و قابلیت‌های کنترل‌کننده (Controller) رابط SDIO معرفی می‌شوند. سپس یک نمونه طراحی کامل از حالت SDIO Slave ارائه می‌شود که شامل موارد زیر است:

- دیاگرام اتصالات سخت‌افزاری (Hardware Connection Diagram)
- روش پیکربندی رجیسترها (Register Configuration)
- سازوکار پاسخ‌دهی به فرمان‌ها (Command Response Mechanism)
- روند ارسال و دریافت داده (Data Transmission and Reception)

جدول ۱ دامنهٔ کاربرد سند

دامنهٔ کاربرد	سری
میکروکنترلرهای همگانی‌منظوره	CH32V307 · CH32H417 · CH32V407

بخش ۱

مرور رابط SDIO در سری CH32

◆ ۱.۱ مرور کلی

رابط SDIO یک رابط ارتباطی روی میکروکنترلر است که برای کار با کارت‌های حافظهٔ خارجی مانند کارت SD یا دستگاه‌های دیگر طراحی شده و یکی از جانبی‌های میکروکنترلر به شمار می‌آید. SDIO میکروکنترلر مستقیماً روی گذرگاه HB نصب می‌شود و ساعت خود را بی‌واسطه از HCLK می‌گیرد؛ به همین دلیل می‌تواند به سرعت ارتباطی بالایی دست یابد. SDIO میکروکنترلر در نقش میزبان SDIO به کار می‌رود و دستگاه تحت کنترل را به‌طور کلی «دستگاه SDIO» یا «دستگاه پیرو SDIO» می‌نامند. محور اصلی این سند راهنمایی کاربر برای پیکربندی MCU به‌عنوان دستگاه پیرو SDIO است تا با دستگاه میزبان ارتباط برقرار کند و انتقال سریع داده را به سرانجام برساند.

سری CH32 به‌عنوان میکروکنترلر بین‌سیستمیِ صنعتی در اکوسیستم RISC-V (چینگ‌که) است و کنترل‌کنندهٔ SDIO آن در نقش دستگاه پیرو صرفاً یک کانال عبور دادهٔ سنتی نیست، بلکه واحد شتاب‌دهندهٔ سخت‌افزاریِ کلیدی برای ساختن سیستم ارتباطی به شمار می‌آید. در طراحی سیستم‌های نهفته با حجم دادهٔ بالا، پردازندهٔ میزبان (مانند SoC سطح کاربرد ARM، یک MCU دیگرِ RISC-V یا آرایهٔ منطقی FPGA) معمولاً باید بسته‌های داده و دستورهای کنترل لحظه‌ای را با توان عملیاتی بالا، تأخیر کم و بدون دخالت CPU با سری CH32 جابه‌جا کند و جابه‌جایی داده مستقیماً در سطح رجیستر و گذرگاه انجام می‌شود. در کنار کنترل‌کنندهٔ DMA سری CH32 و توان انتقال دادهٔ چندبعدیِ آن، حالت دستگاه پیرو از مسیر سازوکار همکاری DMA و وقفه، بدون مشارکت فعال هسته، زنجیرهٔ کامل از دریافت و جابه‌جایی داده تا پاسخ‌دهی را می‌بندد و بازده انتقال در لایهٔ فیزیکی را به حد نهایی رابط می‌رساند. طراحی سخت‌افزاری آن به‌دقت از مشخصات SDIO پیروی می‌کند و به همین دلیل می‌تواند با انواع میزبان‌های منطبق با پروتکل ارتباط برقرار کند.



شکل ۱-۱ دیاگرام ارتباط میزبان و دستگاه پیرو SDIO

توجه: رابط SDIO سه پهنای خط دادهٔ ۱ بیتی، ۴ بیتی و ۸ بیتی برای انتخاب دارد.

بخش ۲

نمونهٔ طراحی کامل حالت SDIO Slave

◆ ۲.۱ اتصال سخت‌افزاری

رابط SDIO در بیشترین حالت از ۱۰ خط سیگنال تشکیل می‌شود (در حالت ۸ بیتی از خطوط دادهٔ DATA0-DATA7، در حالت ۴ بیتی از DATA0-DATA3 و در حالت ۱ بیتی تنها از DATA0 استفاده می‌شود). در این نمونهٔ طراحی، حالت ۴ بیتی در نظر گرفته شده است.

۲.۱.۱ اتصال پایه‌ها

در این نمونه، دو CH32V407 به یکدیگر متصل می‌شوند؛ یکی در نقش میزبان و دیگری در نقش دستگاه پیرو، تا ارتباط فرمان و داده میان آنها برقرار شود.

جدول ۲-۱ تخصیص پایه‌های سیگنال

نام سیگنال	جهت	شرح	پایهٔ CH32V407
SDIO_CLK	ورودی	سیگنال ساعت؛ تأمین‌شده از سمت میزبان	PC12
SDIO_CMD	دوطرفه	خط فرمان / پاسخ	PD2
SDIO_D0	دوطرفه	خط دادهٔ ۰	PC8
SDIO_D1	دوطرفه	خط دادهٔ ۱	PC9
SDIO_D2	دوطرفه	خط دادهٔ ۲	PC10
SDIO_D3	دوطرفه	خط دادهٔ ۳	PC11

نکات اتصال سخت‌افزاری:

- مقاومت کشش بالا: خطوط CMD و DATA در حالت بیکار باید در سطح منطقی بالا نگه داشته شوند؛ معمولاً در سمت برد میزبان مقاومت کشش بالای 10kΩ تا 50kΩ پیکربندی می‌شود تا در زمان بیکاری گذرگاه، وضعیت خطوط سیگنال پایدار بماند.
- تطبیق سطح منطقی: سطح I/O برد CH32V407 بر مبنای دامنهٔ تغذیهٔ VDDIO است. اگر ولتاژ I/O میزبان با VDDIO برد CH32V407 (معمولاً 3.3V) یکسان نباشد، باید مدار تبدیل سطح منطقی افزوده شود.
- یکپارچگی سیگنال: رابط SDIO در حالت ۴ خطی به نرخ انتقال 80MHz می‌رسد؛ در ارتباط پرسرعت طراحیِ تطبیق امپدانس توصیه می‌شود، از اتصال با سیم‌های پل (دوپونت) پرهیز کنید و روش اتصال مستقیم کوتاه‌مسیر روی PCB توصیه می‌شود.

۲.۲ پیکربندی رجیسترها

میزبان و دستگاه پیرو به‌صورت جداگانه پیکربندی می‌شوند. گذرگاه SDIO از اصل ارتباطیِ «میزبان آغاز می‌کند، دستگاه پیرو پاسخ می‌دهد» پیروی می‌کند و همهٔ عملیات انتقال داده به‌واسطهٔ ارسال فرمان از سمت میزبان آغاز می‌شود.

۲.۲.۱ پیکربندی میزبان

پیکربندی میزبان به این شکل است و شامل مقداردهی اولیهٔ پایه‌های IO، ساعت، پهنای داده، لبهٔ نمونه‌برداری و دیگر موارد می‌شود:

```
SD_Init( void )
{
GPIO_InitTypeDef  GPIO_InitStructure;
    SD_Error errorstatus = SD_OK;

    RCC_PB2PeriphClockCmd(RCC_PB2Periph_GPIOC | RCC_PB2Periph_GPIOD, ENABLE );
    RCC_HBPeriphClockCmd( RCC_HBPeriph_SDIO, ENABLE );
    RCC_PB2PeriphClockCmd( RCC_PB2Periph_AFIO, ENABLE );

    GPIO_InitStructure.GPIO_Pin =  GPIO_Pin_10 | GPIO_Pin_11 | GPIO_Pin_12;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOC, &GPIO_InitStructure );

    GPIO_InitStructure.GPIO_Pin =  GPIO_Pin_8 | GPIO_Pin_9 ;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOC, &GPIO_InitStructure );

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOD, &GPIO_InitStructure );

    SDIO_DeInit();

    errorstatus = SD_PowerON();

    return errorstatus;
 }

SD_Error SD_PowerON( void )
{

    SD_Error errorstatus = SD_OK;
    SDIO_InitStructure.SDIO_ClockDiv = 0;   //تنظیم ضریب تقسیم: SDIOCLK = HCLK/(ClockDiv+2)
    SDIO_InitStructure.SDIO_ClockEdge = SDIO_ClockEdge_Falling; //تنظیم لبهٔ نمونه‌برداری
    SDIO_InitStructure.SDIO_ClockBypass = SDIO_ClockBypass_Disable; //فعال یا غیرفعال‌کردن میان‌بری ساعت
    SDIO_InitStructure.SDIO_ClockPowerSave = SDIO_ClockPowerSave_Disable;//بستن ساعت در حالت بیکار
    SDIO_InitStructure.SDIO_BusWide = SDIO_BusWide_4b;//پهنای داده
    SDIO_InitStructure.SDIO_HardwareFlowControl = SDIO_HardwareFlowControl_Disable;//جریان سخت‌افزاری
    SDIO_Init( &SDIO_InitStructure );

    SDIO_SetPowerState( SDIO_PowerState_ON );//تنظیم به حالت برق‌رسانی‌شده

    SDIO_ClockCmd(Enable ); //روشن‌کردن ساعت

return( errorstatus );
}
```

کد ۱ مقداردهی اولیهٔ پایه‌ها، ساعت و کنترل‌کنندهٔ SDIO در میزبان

پس از پایان مقداردهی اولیه، میزبان ارسال CMD را آغاز می‌کند و به دستگاه پیرو اعلام آمادگی می‌دهد؛ سپس منتظر پاسخ دستگاه پیرو می‌ماند. دستگاه پیرو تنها از پاسخ از نوع R1 پشتیبانی می‌کند:

```
while ((SDIO->STA&SDIO_FLAG_CMDREND)==0)
{
 SDIO_CmdInitStructure.SDIO_Argument = 0x6; //ارسالی (ARG) آرگومان
SDIO_CmdInitStructure.SDIO_CmdIndex = 0x05;//ارسالی (CMD) فرمان
 SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short; //نوع پاسخ در انتظار
 SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
 SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;//فعال‌سازی ارسال
 SDIO_SendCommand( &SDIO_CmdInitStructure );
}
```

کد ۲ ارسال فرمان 0x05 و انتظار پاسخ در میزبان

پس از تغییر ماشین حالت و رسیدن وضعیت میزبان به CMDREND، پاسخ دریافت شده و آزمون CRC با موفقیت انجام شده است؛ میزبان ارسال داده را آغاز می‌کند.

```
//تابع پیکربندی DMA
 void SD_DMA_Config( u32 *mbuf, u32 bufsize, u32 DMA_DIR )
{
    DMA_InitTypeDef DMA_InitStructure;

    RCC_HBPeriphClockCmd( RCC_HBPeriph_DMA2, ENABLE );

    DMA_DeInit( DMA2_Channel4 );
    DMA_Cmd( DMA2_Channel4, DISABLE );

    DMA_InitStructure.DMA_PeripheralBaseAddr = ( u32 )&SDIO->FIFO;
    DMA_InitStructure.DMA_MemoryBaseAddr = ( u32 )mbuf;
    DMA_InitStructure.DMA_DIR = DMA_DIR;
    DMA_InitStructure.DMA_BufferSize = bufsize/4 ;
    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;
    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Word;
    DMA_InitStructure.DMA_Mode = DMA_Mode_Normal;
    DMA_InitStructure.DMA_Priority = DMA_Priority_High;
    DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;
    DMA_Init( DMA2_Channel4, &DMA_InitStructure );
}
//تابع ارسال میزبان
Void HOST_Send()
{
SD_DMA_Config(( u32 * )buf,512,DMA_DIR_PeripheralDST);//فراخوانی پیکربندی DMA با پارامترهای انتقال
    SDIO_DataInitStructure.SDIO_DataBlockSize = 9 << 4; ; //512 :طول بلوک دادهٔ انتقال B
    SDIO_DataInitStructure.SDIO_DataLength = 512 ;
    SDIO_DataInitStructure.SDIO_DataTimeOut = SD_DATATIMEOUT ;//تنظیم مهلت زمانی
    SDIO_DataInitStructure.SDIO_DPSM = SDIO_DPSM_Enable;
    SDIO_DataInitStructure.SDIO_TransferDir = SDIO_TransferDir_ToCard; //جهت انتقال
    SDIO_DataInitStructure.SDIO_TransferMode = SDIO_TransferMode_Block;//انتقال در حالت بلوکی
SDIO_DataConfig( &SDIO_DataInitStructure );

    SDIO_DMACmd( ENABLE );//فعال‌سازی DMA
    DMA_Cmd( DMA2_Channel4, ENABLE );
    while( ( ( DMA2->INTFR & 0X2000 ) == RESET ) ) //انتظار پرچم پایان انتقال DMA
```

```
    {
    }
    while(((SDIO->STA&SDIO_FLAG_DBCKEND)==0)) //انتظار پرچم ارسال یا دریافت بلوک داده همراه با درستی CRC
    {
    }
    SDIO->DCTRL&=~(1<<0); //بستن ارسال SDIO

    SDIO_DMACmd( DISABLE );
    DMA_Cmd( DMA2_Channel4, DISABLE );//بستن DMA
    printf("data send OK!!!\r\n");
SDIO->ICR=0xFFFFFF; //پاکسازی بیت‌های وضعیت SDIO
}
```

کد ۳ پیکربندی DMA و تابع ارسال داده در میزبان

◆ ۲.۲.۲ پیکربندی برده

پیکربندی برده به این شکل است و شامل مقداردهی اولیهٔ پایه‌های IO، پهنای داده و دیگر موارد می‌شود:

```
SD_Init( void )
{
GPIO_InitTypeDef  GPIO_InitStructure;
    SD_Error errorstatus = SD_OK;

    RCC_PB2PeriphClockCmd(RCC_PB2Periph_GPIOC | RCC_PB2Periph_GPIOD, ENABLE );
    RCC_HBPeriphClockCmd( RCC_HBPeriph_SDIO, ENABLE );
    RCC_PB2PeriphClockCmd( RCC_PB2Periph_AFIO, ENABLE );

    GPIO_InitStructure.GPIO_Pin =  GPIO_Pin_10 | GPIO_Pin_11 | GPIO_Pin_12;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOC, &GPIO_InitStructure );

    GPIO_InitStructure.GPIO_Pin =   GPIO_Pin_8 | GPIO_Pin_9 ;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOC, &GPIO_InitStructure );

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOD, &GPIO_InitStructure );

    SDIO_DeInit();

    errorstatus = SD_PowerON();

    return errorstatus;
 }

SD_Error SD_PowerON( void )
{

    SD_Error errorstatus = SD_OK;
    SDIO_InitStructure.SDIO_ClockDiv = 0;   //تنظیم ضریب تقسیم: SDIOCLK = HCLK/(ClockDiv+2)
    SDIO_InitStructure.SDIO_ClockEdge = SDIO_ClockEdge_Falling; //تنظیم لبهٔ نمونه‌برداری
```

```
    SDIO_InitStructure.SDIO_ClockBypass = SDIO_ClockBypass_Disable; //فعال یا غیرفعال‌کردن میان‌بری ساعت
    SDIO_InitStructure.SDIO_ClockPowerSave = SDIO_ClockPowerSave_Disable;//بستن ساعت در حالت بیکار
    SDIO_InitStructure.SDIO_BusWide = SDIO_BusWide_4b;//پهنای داده
    SDIO_InitStructure.SDIO_HardwareFlowControl = SDIO_HardwareFlowControl_Disable;//جریان سخت‌افزاری
    SDIO_Init( &SDIO_InitStructure );

    SDIO_SetPowerState( SDIO_PowerState_ON );//تنظیم به حالت برق‌رسانی‌شده

SDIO_ClockCmd(Enable ); //روشن‌کردن ساعت

SDIO->DCTRL2|=(1<<24);////فعال‌سازی حالت SLV
    return( errorstatus );
}
```

کد ۴ مقداردهی اولیهٔ برده و فعال‌سازی حالت SLV

پس از پایان مقداردهی اولیه، برده پاسخ R1 برای CMD را پیکربندی می‌کند تا پس از دریافت CMD میزبان به‌صورت خودکار پاسخ دهد، آمادگی خود را به میزبان اعلام کند و وارد حالت انتظار برای ارسال CMD از سمت میزبان شود.

```
SDIO->ARG=0x13;//////////پاسخ R1 0 :به میزبانx13
while ((SDIO->STA&SDIO_FLAG_CMDSENT)==0)
{
}
```

کد ۵ پیکربندی پاسخ R1 و انتظار فرمان در برده

پس از تغییر ماشین حالت و رسیدن وضعیت برده به CMDREND، پاسخ دریافت شده است؛ برده دریافت داده را آغاز می‌کند.

```
//تابع پیکربندی DMA
void SD_DMA_Config( u32 *mbuf, u32 bufsize, u32 DMA_DIR )
{
    DMA_InitTypeDef DMA_InitStructure;

    RCC_HBPeriphClockCmd( RCC_HBPeriph_DMA2, ENABLE );

    DMA_DeInit( DMA2_Channel4 );
    DMA_Cmd( DMA2_Channel4, DISABLE );

    DMA_InitStructure.DMA_PeripheralBaseAddr = ( u32 )&SDIO->FIFO;
    DMA_InitStructure.DMA_MemoryBaseAddr = ( u32 )mbuf;
    DMA_InitStructure.DMA_DIR = DMA_DIR;
    DMA_InitStructure.DMA_BufferSize = bufsize/4 ;
    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;
    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Word;
    DMA_InitStructure.DMA_Mode = DMA_Mode_Normal;
    DMA_InitStructure.DMA_Priority = DMA_Priority_High;
    DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;
    DMA_Init( DMA2_Channel4, &DMA_InitStructure );
```

```
}
//تابع ارسال میزبان
Void SLV_Rec()
{
    SDIO_DataInitStructure.SDIO_DataBlockSize = 9 << 4; ; //512 :طول بلوک دادهٔ انتقال B
    SDIO_DataInitStructure.SDIO_DataLength = 512 ;
    SDIO_DataInitStructure.SDIO_DataTimeOut = SD_DATATIMEOUT ;
    SDIO_DataInitStructure.SDIO_DPSM = SDIO_DPSM_Enable;
    SDIO_DataInitStructure.SDIO_TransferDir = SDIO_TransferDir_ToSDIO; //جهت انتقال: دریافت
    SDIO_DataInitStructure.SDIO_TransferMode = SDIO_TransferMode_Block;//انتقال در حالت بلوکی
SDIO_DataConfig( &SDIO_DataInitStructure );

SD_DMA_Config(( u32 * )Readbuf,512,DMA_DIR_PeripheralSRC);

    SDIO_DMACmd( ENABLE );//فعال‌سازی DMA
    DMA_Cmd( DMA2_Channel4, ENABLE );
    while( ( ( DMA2->INTFR & 0X2000 ) == RESET ) ) //انتظار پرچم پایان انتقال DMA
    {
    }
    while(((SDIO->STA&SDIO_FLAG_DBCKEND)==0)) //انتظار پرچم ارسال یا دریافت بلوک داده همراه با درستی CRC
    {
    }

    SDIO->DCTRL&=~(1<<0); //بستن دریافت SDIO
    SDIO_DMACmd( DISABLE );
    DMA_Cmd( DMA2_Channel4, DISABLE );//بستن DMA
    printf("data send OK!!!\r\n");
SDIO->ICR=0xFFFFFF; //پاک‌سازی بیت‌های وضعیت SDIO
}
```

کد ۶ پیکربندی DMA و تابع دریافت داده در برده

پس از انجام پیکربندی‌های بالا، یک دور کامل عملیات «ارسال از سمت میزبان و دریافت از سمت برده» قابل انجام است.

بخش ۳

آزمون و راستی‌آزمایی

پس از انجام پیکربندی‌های بالا، توصیه می‌شود کارکرد بردهٔ SDIO را طبق گام‌های زیر به‌صورت مرحله‌به‌مرحله راستی‌آزمایی کنید:

- آزمون اتصال سخت‌افزاری: با اسیلوسکوپ وجود شکل موج سیگنال روی خطوط CMD و DATA را اندازه بگیرید تا اتصال فیزیکی درست میان میزبان و برده تأیید شود.
- آزمون شناسایی فرمان: پس از آنکه برده SDIO_FLAG_CMDSENT را تشخیص داد، مقادیر را چاپ کنید؛ پارامتر فرمان دریافتی در RESP1 و شاخص فرمان دریافتی در RESP2 قرار می‌گیرد و از این راه می‌توانید درستی ارسال CMD را بررسی کنید، یا با تحلیل‌گر منطقی آن را دریافت و کدگشایی کنید.

- آزمون انبوه ارسال و دریافت داده: از سمت میزبان داده ارسال کنید و برده محتوای دادهٔ دریافتی را از طریق پورت سریال چاپ کند تا عملکرد دریافت راستی‌آزمایی شود.

وضعیت STA میزبان و برده را بررسی کنید که خطای غیرعادی گزارش نشده باشد.

پیوست

تاریخچهٔ نسخه‌ها و بیانیه

جدول تاریخچه تاریخچهٔ نسخه‌های سند اصلی AN21000

تاریخ	نسخه	شرح تغییرات
2026/06/25	V1.0	انتشار نسخهٔ اول

بیانیه

این راهنما صرفاً جهت مرجع ارائه شده است و نویسندهٔ سند اصلی می‌تواند بدون اطلاع قبلی و در هر زمان، تغییراتی در محتوای آن اعمال کند. ترجمهٔ فارسی این سند توسط تیم معمار الکترونیک صنعت تهیه شده است؛ برای اطمینان از به‌روزترین نسخه به https://wch.cn مراجعه کنید.

پیوست

واژگان کلیدی (واژه‌نامهٔ فنی)

اصطلاحات فنی سخت‌افزار مطابق سند اصلی به‌صورت انگلیسی حفظ شده‌اند. جدول زیر معادل‌ها و شرح کوتاه هر واژه را برای مرجع سریع جمع می‌کند.

جدول واژگان واژگان کلیدی به‌کاررفته در این جزوه

واژه	یعنی چه؟
SDIO	رابط ارتباطی برای کار با کارت‌های حافظه و دستگاه‌های خارجی؛ جانبیِ میکروکنترلر که مستقیماً روی گذرگاه HB نصب است و از HCLK ساعت می‌گیرد
HB / HCLK	گذرگاه پرسرعت میزبان و ساعت مبنای آن؛ منبع ساعت رابط SDIO و تعیین‌کنندهٔ سرعت ارتباط
Host / Slave	میزبان (Host): سمتِ آغازگر که فرمان می‌فرستد و ساعت را تأمین می‌کند؛ دستگاه پیرو (Slave): دستگاه تحت کنترل که به فرمان‌ها پاسخ می‌دهد
SDIO_CLK	خط ساعت ارتباط که همیشه توسط میزبان تأمین می‌شود؛ در نمونهٔ این سند به پایهٔ PC12 نگاشت شده است
CMD	خط دوطرفهٔ فرمان-پاسخ برای انتقال فرمان‌ها از میزبان و ارسال پاسخ از دستگاه پیرو؛ در نمونه به پایهٔ PD2 نگاشت شده است

واژه	یعنی چه؟
R1 Response	نوع پاسخِ تنها پشتیبانی‌شده در حالت Slave؛ پس از دریافت فرمان میزبان، محتوای رجیستر ARG به‌صورت خودکار به‌عنوان پاسخ ارسال می‌شود
DAT0–DAT3	خطوط داده در حالت ۴ بیتی؛ در حالت ۸ بیتی تا DAT7 گسترش می‌یابد و در حالت ۱ بیتی فقط DAT0 به کار می‌رود
CPSM	ماشین حالت فرمان در کنترل‌کنندهٔ SDIO؛ با فعال‌شدن، فرمان پیکربندی‌شده را روی خط CMD می‌فرستد
DPSM	ماشین حالت داده در کنترل‌کنندهٔ SDIO؛ انتقال داده را در جهت کارت یا در جهت SDIO (دریافت) مدیریت می‌کند
DMA	میسر دسترسی مستقیم به حافظه؛ جابه‌جایی داده میان FIFO کنترل‌کنندهٔ SDIO و حافظه را بدون دخالت هسته انجام می‌دهد
FIFO	میان‌گیر دادهٔ کنترل‌کنندهٔ SDIO؛ آدرس پایهٔ کانال DMA در نمونهٔ این سند
STA Flags	پرچم‌های رجیستر وضعیت: CMDREND پایان فرمان همراه با پاسخ، CMDSENT ارسال کامل فرمان و DBCKEND پایان بلوک داده همراه با درستیِ CRC
DCTRL2	رجیستر کنترل داده در حالت Slave؛ قرار دادن بیت ۲۴ آن، کارکرد دستگاه پیرو (SLV) را فعال می‌کند
VDDIO	دامنهٔ تغذیهٔ پایه‌های ورودی-خروجی؛ مبنای سطح منطقی I/O که در نمونهٔ این سند معمولاً 3.3V است



معمار الکترونیک صنعت

MEMAR ELECTRONIC SANAT (MES)

در مورد مطالب این جزوه و ترجمه، بدیهی است که از نقطه‌نظرات و پیشنهادهای شما عزیزان در این راستا خوشحال می‌شویم. همچنین اشکالات، کم‌وکاستی‌ها و پرسش‌های خود را در این مورد با ما در میان بگذارید تا در نسخه‌های بعدی لحاظ شوند.

با تشکر، حسن معماری

ترجمهٔ فارسی سند AN21000 V1.0 · نسخهٔ ۱٫۰ — تاریخ ساخت: ۱۴۰۵/۰۸/۲۳

راه‌های ارتباط با ما

گیت‌هاب	https://github.com/MEMAR-ELECTRONIC-SANAT
کانال تلگرام	https://t.me/Memar_Electronic
پیج اینستاگرام	https://www.instagram.com/Memar.Electronic
ایمیل	Memari.h@gmail.com